



Choupo

The Open Source Chemical Process Simulator

Developer Guide

Version v0.2.0

18 June 2026

Copyright © 2026 Vítor Geraldês.

Authors: Vítor Geraldês.

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

Code examples, Choupo case files, and other machine-readable examples are licensed under GPL-3.0-or-later.

Typeset in L^AT_EX.

Acknowledgment. Choupo was conceived and architected by Vítor Geraldês. Substantial parts of the initial implementation, documentation, and tutorial corpus were produced with assistance from Anthropic Claude Code using Claude Opus/Fable models. This guide is published under human curation, review, and editorial responsibility by Vítor Geraldês.

License

Attribution-ShareAlike 4.0 International

Creative Commons Corporation ("Creative Commons") is not a law firm and does not provide legal services or legal advice. Distribution of Creative Commons public licenses does not create a lawyer-client or other relationship. Creative Commons makes its licenses and related information available on an "as-is" basis. Creative Commons gives no warranties regarding its licenses, any material licensed under their terms and conditions, or any related information. Creative Commons disclaims all liability for damages resulting from their use to the fullest extent possible.

Using Creative Commons Public Licenses

Creative Commons public licenses provide a standard set of terms and conditions that creators and other rights holders may use to share original works of authorship and other material subject to copyright and certain other rights specified in the public license below. The following considerations are for informational purposes only, are not exhaustive, and do not form part of our licenses.

Considerations for licensors: Our public licenses are intended for use by those authorized to give the public permission to use material in ways otherwise restricted by copyright and certain other rights. Our licenses are irrevocable. Licensors should read and understand the terms and conditions of the license they choose before applying it. Licensors should also secure all rights necessary before applying our licenses so that the public can reuse the material as expected. Licensors should clearly mark any material not subject to the license. This includes other CC-licensed material, or material used under an exception or limitation to copyright. More considerations for licensors: wiki.creativecommons.org/Considerations_for_licensors

Considerations for the public: By using one of our public licenses, a licensor grants the public permission to use the licensed material under specified terms and conditions. If the licensor's permission is not necessary for any reason--for example, because of any applicable exception or limitation to copyright--then that use is not regulated by the license. Our licenses grant only permissions under copyright and certain other rights that a licensor has authority to grant. Use of the licensed material may still be restricted for other reasons, including because others have copyright or other rights in the material. A licensor may make special requests, such as asking that all changes be marked or described. Although not required by our licenses, you are encouraged to respect those requests where reasonable. More considerations for the public: wiki.creativecommons.org/Considerations_for_licensees

Creative Commons Attribution-ShareAlike 4.0 International Public License

By exercising the Licensed Rights (defined below), You accept and agree to be bound by the terms and conditions of this Creative Commons Attribution-ShareAlike 4.0 International Public License ("Public License"). To the extent this Public License may be interpreted as a contract, You are granted the Licensed Rights in consideration of Your acceptance of these terms and conditions, and the Licensor grants You such rights in consideration of benefits the Licensor receives from making the Licensed Material available under these terms and conditions.

Section 1 -- Definitions.

- a. Adapted Material means material subject to Copyright and Similar Rights that is derived from or based upon the Licensed Material and in which the Licensed Material is translated, altered, arranged, transformed, or otherwise modified in a manner requiring permission under the Copyright and Similar Rights held by the Licensor. For purposes of this Public License, where the Licensed Material is a musical work, performance, or sound recording, Adapted Material is always produced where the Licensed Material is synched in timed relation with a moving image.
- b. Adapter's License means the license You apply to Your Copyright and Similar Rights in Your contributions to Adapted Material in accordance with the terms and conditions of this Public License.
- c. BY-SA Compatible License means a license listed at creativecommons.org/compatiblelicenses, approved by Creative Commons as essentially the equivalent of this Public License.
- d. Copyright and Similar Rights means copyright and/or similar rights closely related to copyright including, without limitation, performance, broadcast, sound recording, and Sui Generis Database Rights, without regard to how the rights are labeled or categorized. For purposes of this Public License, the rights specified in Section 2(b)(1)-(2) are not Copyright and Similar

Rights.

- e. Effective Technological Measures means those measures that, in the absence of proper authority, may not be circumvented under laws fulfilling obligations under Article 11 of the WIPO Copyright Treaty adopted on December 20, 1996, and/or similar international agreements.
- f. Exceptions and Limitations means fair use, fair dealing, and/or any other exception or limitation to Copyright and Similar Rights that applies to Your use of the Licensed Material.
- g. License Elements means the license attributes listed in the name of a Creative Commons Public License. The License Elements of this Public License are Attribution and ShareAlike.
- h. Licensed Material means the artistic or literary work, database, or other material to which the Licensor applied this Public License.
- i. Licensed Rights means the rights granted to You subject to the terms and conditions of this Public License, which are limited to all Copyright and Similar Rights that apply to Your use of the Licensed Material and that the Licensor has authority to license.
- j. Licensor means the individual(s) or entity(ies) granting rights under this Public License.
- k. Share means to provide material to the public by any means or process that requires permission under the Licensed Rights, such as reproduction, public display, public performance, distribution, dissemination, communication, or importation, and to make material available to the public including in ways that members of the public may access the material from a place and at a time individually chosen by them.
- l. Sui Generis Database Rights means rights other than copyright resulting from Directive 96/9/EC of the European Parliament and of the Council of 11 March 1996 on the legal protection of databases, as amended and/or succeeded, as well as other essentially equivalent rights anywhere in the world.
- m. You means the individual or entity exercising the Licensed Rights under this Public License. Your has a corresponding meaning.

Section 2 -- Scope.

- a. License grant.
 - 1. Subject to the terms and conditions of this Public License, the Licensor hereby grants You a worldwide, royalty-free, non-sublicensable, non-exclusive, irrevocable license to exercise the Licensed Rights in the Licensed Material to:
 - a. reproduce and Share the Licensed Material, in whole or in part; and
 - b. produce, reproduce, and Share Adapted Material.
 - 2. Exceptions and Limitations. For the avoidance of doubt, where Exceptions and Limitations apply to Your use, this Public License does not apply, and You do not need to comply with its terms and conditions.
 - 3. Term. The term of this Public License is specified in Section 6(a).
 - 4. Media and formats; technical modifications allowed. The Licensor authorizes You to exercise the Licensed Rights in all media and formats whether now known or hereafter created, and to make technical modifications necessary to do so. The Licensor waives and/or agrees not to assert any right or authority to forbid You from making technical modifications necessary to exercise the Licensed Rights, including technical modifications necessary to circumvent Effective Technological Measures. For purposes of this Public License, simply making modifications authorized by this Section 2(a) (4) never produces Adapted Material.
 - 5. Downstream recipients.
 - a. Offer from the Licensor -- Licensed Material. Every recipient of the Licensed Material automatically receives an offer from the Licensor to exercise the Licensed Rights under the terms and conditions of this Public License.
 - b. Additional offer from the Licensor -- Adapted Material. Every recipient of Adapted Material from You automatically receives an offer from the Licensor to exercise the Licensed Rights in the Adapted Material under the conditions of the Adapter's License You apply.
 - c. No downstream restrictions. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, the Licensed Material if doing so restricts exercise of the Licensed Rights by any recipient of the Licensed Material.

6. No endorsement. Nothing in this Public License constitutes or may be construed as permission to assert or imply that You are, or that Your use of the Licensed Material is, connected with, or sponsored, endorsed, or granted official status by, the Licensor or others designated to receive attribution as provided in Section 3(a)(1)(A)(i).

b. Other rights.

1. Moral rights, such as the right of integrity, are not licensed under this Public License, nor are publicity, privacy, and/or other similar personality rights; however, to the extent possible, the Licensor waives and/or agrees not to assert any such rights held by the Licensor to the limited extent necessary to allow You to exercise the Licensed Rights, but not otherwise.
2. Patent and trademark rights are not licensed under this Public License.
3. To the extent possible, the Licensor waives any right to collect royalties from You for the exercise of the Licensed Rights, whether directly or through a collecting society under any voluntary or waivable statutory or compulsory licensing scheme. In all other cases the Licensor expressly reserves any right to collect such royalties.

Section 3 -- License Conditions.

Your exercise of the Licensed Rights is expressly made subject to the following conditions.

a. Attribution.

1. If You Share the Licensed Material (including in modified form), You must:
 - a. retain the following if it is supplied by the Licensor with the Licensed Material:
 - i. identification of the creator(s) of the Licensed Material and any others designated to receive attribution, in any reasonable manner requested by the Licensor (including by pseudonym if designated);
 - ii. a copyright notice;
 - iii. a notice that refers to this Public License;
 - iv. a notice that refers to the disclaimer of warranties;
 - v. a URI or hyperlink to the Licensed Material to the extent reasonably practicable;
 - b. indicate if You modified the Licensed Material and retain an indication of any previous modifications; and
 - c. indicate the Licensed Material is licensed under this Public License, and include the text of, or the URI or hyperlink to, this Public License.
2. You may satisfy the conditions in Section 3(a)(1) in any reasonable manner based on the medium, means, and context in which You Share the Licensed Material. For example, it may be reasonable to satisfy the conditions by providing a URI or hyperlink to a resource that includes the required information.
3. If requested by the Licensor, You must remove any of the information required by Section 3(a)(1)(A) to the extent reasonably practicable.

b. ShareAlike.

In addition to the conditions in Section 3(a), if You Share Adapted Material You produce, the following conditions also apply.

1. The Adapter's License You apply must be a Creative Commons license with the same License Elements, this version or later, or a BY-SA Compatible License.
2. You must include the text of, or the URI or hyperlink to, the Adapter's License You apply. You may satisfy this condition in any reasonable manner based on the medium, means, and context in which You Share Adapted Material.
3. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, Adapted Material that restrict exercise of the rights granted under the Adapter's License You apply.

Section 4 -- Sui Generis Database Rights.

Where the Licensed Rights include Sui Generis Database Rights that apply to Your use of the Licensed Material:

- a. for the avoidance of doubt, Section 2(a)(1) grants You the right to extract, reuse, reproduce, and Share all or a substantial portion of the contents of the database;
- b. if You include all or a substantial portion of the database contents in a database in which You have Sui Generis Database Rights, then the database in which You have Sui Generis Database Rights (but not its individual contents) is Adapted Material, including for purposes of Section 3(b); and
- c. You must comply with the conditions in Section 3(a) if You Share all or a substantial portion of the contents of the database.

For the avoidance of doubt, this Section 4 supplements and does not replace Your obligations under this Public License where the Licensed Rights include other Copyright and Similar Rights.

Section 5 -- Disclaimer of Warranties and Limitation of Liability.

- a. UNLESS OTHERWISE SEPARATELY UNDERTAKEN BY THE LICENSOR, TO THE EXTENT POSSIBLE, THE LICENSOR OFFERS THE LICENSED MATERIAL AS-IS AND AS-AVAILABLE, AND MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND CONCERNING THE LICENSED MATERIAL, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHER. THIS INCLUDES, WITHOUT LIMITATION, WARRANTIES OF TITLE, MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, ABSENCE OF LATENT OR OTHER DEFECTS, ACCURACY, OR THE PRESENCE OR ABSENCE OF ERRORS, WHETHER OR NOT KNOWN OR DISCOVERABLE. WHERE DISCLAIMERS OF WARRANTIES ARE NOT ALLOWED IN FULL OR IN PART, THIS DISCLAIMER MAY NOT APPLY TO YOU.
- b. TO THE EXTENT POSSIBLE, IN NO EVENT WILL THE LICENSOR BE LIABLE TO YOU ON ANY LEGAL THEORY (INCLUDING, WITHOUT LIMITATION, NEGLIGENCE) OR OTHERWISE FOR ANY DIRECT, SPECIAL, INDIRECT, INCIDENTAL, CONSEQUENTIAL, PUNITIVE, EXEMPLARY, OR OTHER LOSSES, COSTS, EXPENSES, OR DAMAGES ARISING OUT OF THIS PUBLIC LICENSE OR USE OF THE LICENSED MATERIAL, EVEN IF THE LICENSOR HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH LOSSES, COSTS, EXPENSES, OR DAMAGES. WHERE A LIMITATION OF LIABILITY IS NOT ALLOWED IN FULL OR IN PART, THIS LIMITATION MAY NOT APPLY TO YOU.
- c. The disclaimer of warranties and limitation of liability provided above shall be interpreted in a manner that, to the extent possible, most closely approximates an absolute disclaimer and waiver of all liability.

Section 6 -- Term and Termination.

- a. This Public License applies for the term of the Copyright and Similar Rights licensed here. However, if You fail to comply with this Public License, then Your rights under this Public License terminate automatically.
- b. Where Your right to use the Licensed Material has terminated under Section 6(a), it reinstates:
 1. automatically as of the date the violation is cured, provided it is cured within 30 days of Your discovery of the violation; or
 2. upon express reinstatement by the Licensor.

For the avoidance of doubt, this Section 6(b) does not affect any right the Licensor may have to seek remedies for Your violations of this Public License.

- c. For the avoidance of doubt, the Licensor may also offer the Licensed Material under separate terms or conditions or stop distributing the Licensed Material at any time; however, doing so will not terminate this Public License.
- d. Sections 1, 5, 6, 7, and 8 survive termination of this Public License.

Section 7 -- Other Terms and Conditions.

- a. The Licensor shall not be bound by any additional or different terms or conditions communicated by You unless expressly agreed.
- b. Any arrangements, understandings, or agreements regarding the Licensed Material not stated herein are separate from and independent of the terms and conditions of this Public License.

Section 8 -- Interpretation.

- a. For the avoidance of doubt, this Public License does not, and shall not be interpreted to, reduce, limit, restrict, or impose conditions on any use of the Licensed Material that could lawfully be made without permission under this Public License.
- b. To the extent possible, if any provision of this Public License is deemed unenforceable, it shall be automatically reformed to the minimum extent necessary to make it enforceable. If the provision cannot be reformed, it shall be severed from this Public License without affecting the enforceability of the remaining terms and conditions.

- c. No term or condition of this Public License will be waived and no failure to comply consented to unless expressly agreed to by the Licensor.
- d. Nothing in this Public License constitutes or may be interpreted as a limitation upon, or waiver of, any privileges and immunities that apply to the Licensor or You, including from the legal processes of any jurisdiction or authority.

=====
Creative Commons is not a party to its public licenses. Notwithstanding, Creative Commons may elect to apply one of its public licenses to material it publishes and in those instances will be considered the "Licensor." The text of the Creative Commons public licenses is dedicated to the public domain under the CC0 Public Domain Dedication. Except for the limited purpose of indicating that material is shared under a Creative Commons public license or as otherwise permitted by the Creative Commons policies published at creativecommons.org/policies, Creative Commons does not authorize the use of the trademark "Creative Commons" or any other trademark or logo of Creative Commons without its prior written consent including, without limitation, in connection with any unauthorized modifications to any of its public licenses or any other arrangements, understandings, or agreements concerning use of licensed material. For the avoidance of doubt, this paragraph does not form part of the public licenses.

Creative Commons may be contacted at creativecommons.org.

Trademarks

Choupo is a trademark of TalentGround Lda.

Anthropic, Claude, and Claude Code are trademarks or registered trademarks of Anthropic, PBC.

Linux is a registered trademark of Linus Torvalds.

Python is a trademark or registered trademark of the Python Software Foundation.

Contents

Preface — the contract this code makes with you	1
1 Philosophy — the contract we promise	2
2 Repository tour	3
3 Build system	4
4 Build architecture: shared library, optional modules, case code	5
4.1 The shared engine library (<code>libchoupo.so</code>)	5
4.2 Optional modules — a clean modular split	5
4.3 Case-local code (<code>case/code/</code>)	6
5 Core types	9
5.1 Dictionary	9
5.2 ProcessStream	9
5.3 SimulationResult	9
5.4 ThermoPackage	10
6 The factory pattern — explicit, no macros	11
7 Adding a new unit operation	12
7.1 Step 1 — Skeleton	12
7.2 Step 2 — Implementation	12
7.3 Step 3 — Register	13
7.4 Step 4 — Tutorial	13
7.5 Step 5 — Regression	13
8 Adding a new thermodynamic model	14
9 Property operations (<code>choupoProps</code>)	15
10 Adding components, materials, reactions	16
10.1 Components	16
10.2 Materials	18
10.3 Utilities	18
10.4 Reactions	18
11 Adding an outer driver / post-processor	20
11.1 Outer driver	20
11.2 Post-processor	20
12 Adding a sizing model	21
13 Coding conventions	22
13.1 Style	22
13.2 Includes	22
13.3 Errors	22
13.4 Comments	22
13.5 Header banner	22
14 Regression testing	23
15 Debugging	24
15.1 Debug build	24
15.2 Verbosity	24
15.3 Sanitizers	24
15.4 The most common failure: NaN propagation	24
16 Architecture deep-dive — the three layers	25
17 Roadmap	26

18 Things to NOT do	27
19 Quick reference	28

Preface — the contract this code makes with you

If you have written CFD glue for a CFD solver, or stared too long at a black-box solver, you already know why this project exists. Every Newton step is auditable. Every dispatch is explicit. Every dependency is in the C++ standard library. The price of that discipline is that contributors have to play by a few rules. They are short, and they are non-negotiable.

This guide is opinionated on purpose. Read §1 carefully — those three principles override everything else — and treat §18 as the project's only blunt list of don'ts.

1 Philosophy — the contract we promise

What you'll learn. The three invariants that every contribution is judged against, in plain language.

1. **Pedagogical clarity beats elegance.** The simulator must be readable by a chemical-engineering student in their third year. No template metaprogramming, no auto-magic, no clever short-circuits. Every Newton iteration and every K -value must be inspectable in both the source and the run-time output.
2. **Source transparency — no hidden registration.** Every factory base class has an *explicit* `registerBuiltins()` called from `main.cpp`. No `REGISTER_TYPE(MyClass)` macros, no static initialisers, no linker-dependent constructors. The student grep-searches `registerBuiltins` and *sees* every built-in type.
3. **Dependency purity — no copyleft or restrictive libraries linked into the binary.** C++ 17, the C++ standard library, and GNU make. Nothing else. No Boost, no Eigen, no Sundials, no CMake. Hand-rolled Newton, Gauss elimination, RK4, Wegstein, Michelsen.

Key idea. An “industrial-grade” contribution that violates any of these will be rejected even if it converges better and runs faster.

2 Repository tour

What you'll learn. What lives where. Useful as a map; you'll come back to it.

```

Choupo/
|-- Makefile                top-level: release/debug switch
|-- make/
|   |-- compiler.mk         CXX, CXXFLAGS, PLATFORM detection
|   |-- rules.mk            pattern rules + dependency tracking
|   '-- wasm.mk             Emscripten WASM build (gui/)
|-- build/<PLATFORM>[Debug]/ all.o/.d/binary (gitignored)
|-- choupoSolve             symlink -> last built binary
|-- etc/bashrc              sourceable shell env
|-- bin/                    runCase, runApplication, listCases, cleanCase
|-- data/
|   |-- components/         <name>.dat --- component database
|   |-- materials/         <name>.dat --- material database
|   '-- README.md           data-file format
|-- docs/                   THIS FILE + userGuide.{md,tex,pdf}
|-- src/
|   |-- core/               Dictionary, SimulationResult, Constants, Types
|   |-- streams/            ProcessStream
|   |-- thermo/             Component, ThermoPackage, Database +
|   |                       phase/, vaporPressure/, activityCoefficient/,
|   |                       equationOfState/, heatCapacity/
|   |-- materials/          Material, MaterialRegistry
|   |-- solver/             NewtonRaphson, NewtonND, Wegstein, StabilityTest
|   |-- unitOperations/     UnitOperation base + per-category folders
|   |-- outerDriver/        OuterDriver base + SweepDriver + FitBinaryPair
|   |-- postProcessing/     PostProcessor base + Sizing/Costing passes
|   |-- propertyOps/        PropertyOperation base (choupoProps)
|   |-- control/            Controller base + PID/Schedule (choupoCtrl)
|   '-- applications/       choupoSolve / choupoBatch / choupoCtrl /
|                           choupoProps -- one main.cpp per binary
|-- tests/                  regression scripts (bin/runTests)
|-- tutorials/              end-to-end cases (104 at, under
|                           steady/ batch/ ctrl/ props/ plant/)
|-- gui/                    web GUI (see CLAUDE.md Sec. 12)
|-- LICENSE                 GPL-3.0-or-later
'-- README.md

```

Key idea. There is *one binary per problem class*, four in all — they share the same library under `src/{core,thermo,solver,...}` and differ only in their `applications/<name>/main.cpp`:

- `choupoSolve` — steady-state simulation, $F(\mathbf{x}) = 0$ (root-finding);
- `choupoBatch` — batch / recipe-driven, per-vessel ODE integration + time-triggered events;
- `choupoCtrl` — dynamic continuous + control, $d\mathbf{Y}/dt = f(\mathbf{Y}, \mathbf{u}, t)$ with PID/schedule controllers;
- `choupoProps` — property evaluation + parameter fitting (a distinct problem class: neither root-finding nor integration).

We split by *problem class*, never by numerical *strategy* (no **Foam*-style proliferation): within each binary the strategies (Newton, Wegstein, RK4, Nelder–Mead, ...) are run-time selections via dicts, not compile-time ones.

3 Build system

```
make all           # release, -O2, build/linux64Gcc/  
make MODE=debug all # -O0 -g3, build/linux64GccDebug/  
make print        # show resolved variables  
make clean        # rm objs + binary for current MODE  
make distclean    # rm entire build/
```

Sources are discovered with `find src -name '*.cpp'` — drop a new `.cpp` anywhere under `src/` and it picks up automatically. Header dependencies are auto-tracked via `-MMD -MP` (`.d` files alongside each `.o`).

Platform string is derived from `uname -s -m`. Currently only `linux64Gcc` is exercised; `darwin64Clang` and `linux64Clang` should “just work” with the same Makefile (modulo `<filesystem>` linking on older toolchains — see §15).

The WASM build (`make wasm`) is a sibling target — see `make/wasm.mk` and `CLAUDE.md` §13 for the Emscripten quirks.

4 Build architecture: shared library, optional modules, case code

Since the build is *dynamic* and *modular*. Three forces drove it: incremental compilation (change one binary, don't rebuild the engine), a clean extension story (users add unit ops without touching `src/`), and — load-bearing for the modular architecture — making it hard to produce a standalone binary that carries the whole engine inside. The model: an engine built as a shared library, and small binaries that *reference* it.

Key idea. A binary never embeds the engine; it links `libchoupo.so` at build time and finds it at run time via `rpath`. A binary copied to a machine without the library simply does not start. This is the same model — one engine, many small binaries that share it, so rebuilding a binary never rebuilds the engine.

4.1 The shared engine library (`libchoupo.so`)

`make lib` archives every engine object (everything under `src/` except the per-application entry points) into a single shared object:

```
make lib                # -> build/<plat>/libchoupo.so    ($(CXX) -shared)
```

The engine is compiled `-fPIC` (position-independent) on Linux. Each binary then links `-lchoupo` with `rpath` set to `$ORIGIN` — the binary's own directory — which is where the `.so` is built; the project-root symlinks resolve to the real binary location, so `$ORIGIN` still points at the `.so`. The result is tiny binaries:

Artefact	Size	Contains the engine?
<code>libchoupo.so</code>	1.7 MB	it <i>is</i> the engine
<code>choupoSolve</code> (binary)	86 KB	no — references it
<code>choupoCase</code> (a user's)	~100 KB	no — references it

Common pitfall. The `win64MinGW` cross-build keeps *static* linking on purpose: the generated `.exe` is self-contained. The `LIB_DEP` / `LIB_LINK` Makefile variables carry the platform branch (objects directly for `win64MinGW`, `-lchoupo` on Linux).

4.2 Optional modules — a clean modular split

The engine can carry OPTIONAL modules as separate `.so` files, so a slim core (the teaching engine) builds without them. The membrane module is Vitor's NF/RO research code — conceptually distinct from the core and kept in its own library so a core-only build is possible. Everything is GPL-3.0-or-later; the split is modular, not a licence boundary:

Library	Contents	Licence
<code>libchoupo.so</code>	core (thermo, solver, unit ops, reports)	GPL-3.0-or-later
<code>libchoupoMembrane.so</code>	NF/RO membrane (<code>spiralWoundModule</code>)	GPL-3.0-or-later

The mechanism is the same link-time stub-swap used for user code (§4.3), applied to a module:

1. The optional unit op lives under its own folder (`src/unitOperations/membrane/`); its objects are filtered *out* of `LIB_OBJS` and compiled into `libchoupoMembrane.so` instead.
2. It is **not** registered in `UnitOperation::registerBuiltins()` (the core must not depend on it). Instead a `registerMembrane.cpp` in the module defines a `registerModules()` hook that registers the type via the usual explicit factory.
3. `choupoSolve/main.cpp` calls `registerModules()` right after `registerBuiltins()`. The core binary links an empty `registerModulesStub.cpp` *only* when the module is absent; the Makefile (`MEMBRANE_PRESENT := $(wildcard src/unitOperations/membrane)`) links exactly the `.so` *or* the stub, never both.

```
# slim core-only build: remove the module source, rebuild
rm -rf src/unitOperations/membrane && make all
# -> only libchoupo.so is produced; the membrane type is unknown:
#    "unknown type 'spiralWoundModule'"
```

To add another optional module, do exactly what the membrane does: a folder under `src/`, kept out of `registerBuiltins()`, with its own `registerModules()` compiled into its own `.so`. (When a *second* module appears, generalise the single `registerModules()` hook into a small list — still explicit, still no auto-registration.)

4.3 Case-local code (case/code/)

A user — a student, a customer — may write their own C++ under a case's `code/` folder and have it compiled into a per-case binary. Build-time only: **no runtime dlopen**, **no macro self-registration**, faithful to the glass-box credo.

```
case/
  code/
    MyOp.H  MyOp.cpp      # the user's unit op (derives UnitOperation)
    registerUserTypes.cpp # defines registerUserTypes() -> registerType("myOp",...)
    options                                # OPTIONAL: extra -I / -l / flags (see below)
  system/  constant/
```

What code/ can register. The primary, tutorialised use is a new *unit operation*. But `registerUserTypes()` runs *after* every `registerBuiltins()`, so it can register **any type with an explicit factory** — the same hook, the same pattern:

What	via	Status
unit operation	<code>UnitOperation::registerType</code>	demonstrated (userOp01/02)
thermo model (γ , EoS, P^{sat} , c_p)	<code>ActivityModel::registerModel</code> , ...	possible, no tutorial yet
outer driver (sweep / optim / DesignSpec)	<code>OuterDriver::registerType</code>	possible
report / sizing / costing	<code>Report::registerType</code> , ...	possible

What the user writes. Three files in `code/`: the unit op (`MyOp.{H,cpp}`, a normal class deriving `UnitOperation` with a `solve()` override), and `registerUserTypes.cpp`, which puts the class in the factory:

```
// case/code/registerUserTypes.cpp
void registerUserTypes() {
    Choupo::UnitOperation::registerType(
        "myOp", [] { return std::make_unique<Choupo::MyOp>(); });
}
```

How it is compiled. `bin/buildCode` runs one ordinary `g++` (it prints the exact command — nothing hidden):

```
g++ -std=c++17 -O2 -I src \           # (1) the engine's headers
    build/<plat>/.../choupoSolve/main.o \ # (2) the engine main() (calls
                                           # registerUserTypes)
    case/code/registerUserTypes.cpp \   # (3) the user's registration
    case/code/MyOp.cpp \               # (4) the user's unit op
    -Lbuild/<plat> -lchoupo [-lchoupoMembrane] \ # (5) engine.so (+ optional module)
    -Wl,-rpath,build/<plat> \          # (6) where to find the.so at run time
    -o case/code/.build/choupoCase     # (7) the per-case binary (~100 KB)
```

The user's `.cpp` are compiled fresh each time (their `.o` are temporaries in `/tmp`, dropped after linking — only the binary remains). The binary links the engine `.so` DYNAMICALLY (5): it references the engine, never embeds it.

The trick — who supplies `registerUserTypes()`. The engine's `main.o` calls `registerUserTypes()` (right after `registerBuiltins()`) but does *not* define it — an unresolved symbol. The linker resolves it from exactly one place, and that place is the build-time choice:

Binary	<code>registerUserTypes()</code> from	Result
stock <code>choupoSolve</code>	<code>registerUserTypesStub.cpp</code> (empty)	registers nothing
the case's <code>choupoCase</code>	<code>case/code/registerUserTypes.cpp</code>	registers <code>myOp</code>

In the `buildCode` command above the stub is *not* linked — only the user's file is — so the linker resolves the symbol with the user's definition. Plain symbol resolution, no weak symbols, no `dlopen`, no macros. (Optional modules use the same pattern with `registerModules()`, §4.2.)

How it is used — at run time.

```
choupoCase runs:
main()
-> UnitOperation::registerBuiltins() // cstr, flash, mixer,...
-> registerModules()                // optional modules (membrane.so)
-> registerUserTypes()             // THE USER'S: registry()["myOp"] = constructor
Flowsheet reads type myOp; in flowsheetDict
-> UnitOperation::New("myOp")       // look up in the std::map
-> the user's constructor -> object -> the user's solve()
```

The factory is just a `std::map<string, function>`; `registerType` adds an entry, `New` looks one up. `myOp` sits beside the built-in types — no special case.

What `solve()` receives — streams and thermo. The op's whole world arrives through two arguments:

```
int solve(const DictPtr& dict, const ThermoPackage& thermo, int verbosity);
```

Streams (the dict). Choupo is sequential-modular: an op does NOT rummage a global stream registry. The *Flowsheet injects* the unit's inlet stream into the `dict` as sub-dicts, plus the unit's own **operation** block:

```
auto feed = dict->subDict("feed");           // F (kmol/s SI), T, P of the inlet
auto comp = dict->subDict("composition");    // mole fractions z_i
auto oper = dict->subDict("operation");      // YOUR parameters (from flowsheetDict)
```

You read your inputs, compute, and publish outputs into `produced_`; the *Flowsheet* binds them to the next units by the **outputs** (...) order. (For several inlets, the unit declares **inputs** (a b) and reads each.)

Thermodynamics (the thermo). The `ThermoPackage&` gives full access to whatever models the case configured — you do not need to know *which* (γ = NRTL? EoS = SRK?); you call the method and the selected model answers:

```
thermo.n(); thermo.indexOf("benzene"); thermo.comp(i); // count, index, Component
thermo.comp(i).vp().Psat_Pa(T);                       // vapour pressure (Antoine/...)
thermo.Kvec(T, P, x, y);                               // K-values (the case's gamma-phi)
thermo.activity().gamma(T, x);                         // activity coefficients gamma_i
thermo.Hliquid(T, x); thermo.Hvapour(T, y);            // sensible-datum enthalpies
thermo.H_real(T, P, y);                               // real gas (H_ig + EoS departure)
thermo.H_stream_formation(T, P, vf, z);               // elements-datum (carries Hf)
```

So a user op can flash, balance energy, or solve an equilibrium with the case's thermo — exactly what the built-in CSTR / flash / column do; it is a first-class citizen, not an add-on.

Common pitfall. **Transport properties (viscosity, conductivity, diffusivity) do not exist yet** — they are on the roadmap as a `TransportModel` with a factory. When they land they will be reachable through the same `thermo` handle; today a user op has thermodynamics only.

Command	What it does
<code>buildCode <case></code>	<i>compiles only</i> (shows the exact g++ command, reports compiler errors); the development-loop command
<code>runCase <case></code>	<i>runs</i> the case — and if it has a <code>code/</code> folder, compiles it first (visibly) via <code>buildCode</code>

Both step up automatically if invoked from inside the `code/` folder. External dependencies (a property library, GSL, Eigen, ...) go in an optional `code/options` file:

```
# case/code/options
INCLUDE = -I/opt/coolprop/include
LIBS     = -L/opt/coolprop/lib -lCoolProp
FLAGS    = -O3
```

Common pitfall. User code lives in `case/code/`, **never** in `src/`. Three reasons: isolation (the engine tree stays auditable), authorship / licence (`src/` is GPL-3.0-or-later; the user owns their `code/`), and pedagogy (a clear engine-vs-mine boundary). Those external libraries are the *user's* dependencies on the *user's* code — the engine itself stays dependency-free.

5 Core types

What you'll learn. Four types live in `src/core/` and `src/streams/` and propagate through every layer of the simulator: get familiar with them before touching anything else.

5.1 Dictionary

hierarchical plain text. Every case dict is parsed into a `Dictionary`. Key operations:

```
auto d = Dictionary::fromFile("system/flowsheetDict");

scalar T = d->lookupScalar("T");           // raw
scalar P = d->lookupScalar("P", Dims::pressure); // dim-checked
auto sub = d->subDict("operation");
auto units = d->lookupDictList("units");
auto comps = d->lookupWordList("components");

// Path-based mutation --- used by outer drivers
d->setScalarAtPath("units[0].operation.refluxRatio", 3.2);
```

The tokenizer treats `[` and `]` as word chars (so paths work). Optional `FoamFile` headers are skipped silently for compatibility with files from other tools.

Three input forms for scalars. The parser accepts named-unit (`P 1 bar;`), the bracket (`A_w [-1 2 1 0 0] 1.5e-12;`), and raw SI (`R 0.5;`). Named and bracket forms tag the entry with its physical dimensions `[M L T Theta N]` so the dim-checked `lookupScalar(key, expectedDims)` overload can throw a clear error on mismatch. Raw entries pass unchecked — the caller is asserting the value is canonical SI.

Available named dimensions live under `Dims::` in `src/core/Dimensions.H` (`pressure`, `molarFlow`, `massFlow`, `temperature`, `length`, `area`, `volume`, `concentration`, `molarHeatCap`, `viscosity`, `diffusivity`, `permeabilityWater`, `massTransferCoeff`,...). Adopt them at the call sites where the unit op reads its operating parameters.

5.2 ProcessStream

What travels between unit operations (units: SI canonical):

```
struct ProcessStream {
    std::string      name;
    scalar           F;      // kmol/s
    scalar           T;      // K
    scalar           P;      // Pa
    std::vector<scalar> z;    // mole fractions, size = NC
    scalar           vf;     // vapor fraction (VLE outputs)
};
```

Mass-basis flow is a derived view, not a stored field — if you need `kg/s`, call `F_mass(stream, thermo)` from `src/streams/StreamMass.H`, which computes $F \cdot \sum_i z_i MW_i$ on demand. One source of truth for the flow (molar); mass cannot drift out of sync.

When you produce streams in your unit-op, fill *all* fields. At print sites you typically scale by 3600 for `kmol/h` and by 10^{-5} for `bar`, or hand the value plus its dimensions to `DisplayUnits::instance().format(v, dims)` which honours the case's `controlDict.units` preferences.

5.3 SimulationResult

What a single pass produces — input to post-processors and outer drivers:

```
struct SimulationResult {
    std::map<std::string, ProcessStream> streams;
    std::map<std::string, std::map<std::string, scalar>> kpis;
    std::map<std::string, EquipmentSizing> sizings; // SizingPass
    std::map<std::string, CostBreakdown> costs; // CostingPass
    bool converged;
```

```
};
```

The `kpis` map is keyed `unitName → { kpiName → value }`. Outer drivers read it; post-processors append to `sizings / costs`. (The `sizings` field was named `dimensions` until; renamed because `Dimensions` is now the physical-dimensions tag.)

5.4 ThermoPackage

Holds:

- `components` (vector of `Component`),
- one or more `Phase` instances (vapor / liquid / solid),
- convenience getters: `nComponents()`, `component(i)`, `phase(name)`, `Psat(i, T)`, `gamma(phase, x, T)`, etc.

Every unit operation receives a `const ThermoPackage&` in `solve()`.

6 The factory pattern — explicit, no macros

What you'll learn. The single registration mechanism used everywhere in the code, and why we refuse the macro-based alternative.

Every extensible base class follows this template:

```
class UnitOperation {
public:
    using Factory = std::function<std::unique_ptr<UnitOperation>()>;

    static void registerType(const std::string& name, Factory f);
    static std::unique_ptr<UnitOperation> New(const std::string& type);
    static std::vector<std::string> availableTypes();
    static void registerBuiltins();           // <-- called in main()

    // ---- virtual interface ----
    virtual const std::string& type() const = 0;
    virtual int solve(const DictPtr&, const ThermoPackage&, int verbosity) = 0;
    virtual std::vector<ProcessStream> producedStreams() const { return {}; }
    virtual std::map<std::string, scalar> kpis() const { return {}; }

protected:
    // helpers...
};
```

`registerBuiltins()` (in the corresponding `.cpp`) is the single place that *lists every type*:

```
void UnitOperation::registerBuiltins()
{
    auto reg = [](const std::string& n, Factory f) { registerType(n, f); };

    reg("isothermalFlash", []{ return std::make_unique<IsothermalFlash>(); });
    reg("flash",          []{ return std::make_unique<IsothermalFlash>(); });
    reg("adiabaticFlash", []{ return std::make_unique<AdiabaticFlash>(); });
    reg("bubbleT",        []{ return std::make_unique<BubblePoint>(); });
    reg("dewT",           []{ return std::make_unique<DewPoint>(); });
    reg("cstr",           []{ return std::make_unique<CSTR>(); });
    reg("pfr",            []{ return std::make_unique<PFR>(); });
    reg("heatExchanger",  []{ return std::make_unique<HeatExchanger>(); });
    reg("HX",             []{ return std::make_unique<HeatExchanger>(); });
    reg("mixer",          []{ return std::make_unique<Mixer>(); });
    reg("splitter",       []{ return std::make_unique<Splitter>(); });
    reg("distillationColumn", []{ return std::make_unique<DistillationColumn>(); });
    reg("column",         []{ return std::make_unique<DistillationColumn>(); });
}
```

`main.cpp` calls each `registerBuiltins()` once at startup, in order. That is the *entire* registration story. No magic, no surprises.

Common pitfall. Do *not* introduce `REGISTER_TYPE(...)` macros, anonymous-namespace static instances, or any pattern that runs registration as a side effect of TU loading. We have rejected this approach explicitly because the linker can drop unused TUs and the failure mode is silent.

7 Adding a new unit operation

What you'll learn. Five steps and roughly thirty minutes for a simple one — skeleton, implementation, registration, tutorial, regression.

7.1 Step 1 — Skeleton

Create `src/unitOperations/<category>/MyOp.H`:

```
#pragma once
#include "core/Dictionary.H"
#include "thermo/ThermoPackage.H"
#include "unitOperations/UnitOperation.H"

namespace Choupo {

class MyOp : public UnitOperation {
public:
    MyOp() = default;
    const std::string& type() const override { return type_; }

    int solve(const DictPtr& dict,
              const ThermoPackage& thermo,
              int verbosity) override;

    std::vector<ProcessStream> producedStreams() const override { return produced_; }
    std::map<std::string, scalar> kpis() const override { return kpis_; }

private:
    inline static const std::string type_ = "myOp";
    std::vector<ProcessStream> produced_;
    std::map<std::string, scalar> kpis_;
};

} // namespace Choupo
```

7.2 Step 2 — Implementation

`src/unitOperations/<category>/MyOp.cpp` — implement `solve()`. Boilerplate to follow:

```
int MyOp::solve(const DictPtr& dict, const ThermoPackage& thermo, int verbosity)
{
    // 1. Read inputs from dict --- let lookups throw on missing keys
    auto op = dict->subDict("operation");
    const scalar T = op->lookupScalar("T");
    //...

    // 2. Read inlet stream(s)
    const ProcessStream& inlet = inlets_.at(0);

    // 3. Echo inputs (verbosity >= 3)
    if (verbosity >= 3) {
        std::cout << " [MyOp] T = " << T << " K, F_in = " << inlet.F << " kmol/h\n";
    }

    // 4. Solve the model --- use solver/ classes for Newton/Wegstein
    NewtonRaphson nr;
    nr.setVerbose(verbosity >= 3);
    auto root = nr.solve([&](scalar x){ /* g(x) */ return /*...*/; },
                      x0, /*tol=*/1e-8, /*maxIter=*/50);

    // 5. Build produced streams
```

```

    ProcessStream out = inlet;          // copy --- keeps thermo, name overwritten
    out.name = "myOp_out";
    out.T = T;
    out.F = /*... */;
    produced_.push_back(out);

    // 6. Populate KPIs
    kpis_["conversion"] = /*... */;
    kpis_["T_out"]      = T;

    return 0;      // 0 == converged
}

```

7.3 Step 3 — Register

Edit `src/unitOperations/UnitOperation.cpp`:

```

#include "<category>/MyOp.H"
//...
void UnitOperation::registerBuiltin() {
    //...
    reg("myOp", []{ return std::make_unique<MyOp>(); });
}

```

7.4 Step 4 — Tutorial

Create `tutorials/myOp01_quick_smoke/` with `system/controlDict`, `system/flowsheetDict`, and `constant/thermoPackage`. Keep the case trivial so it doubles as a regression test.

7.5 Step 5 — Regression

```

make all
for t in tutorials/*; do
./choupoSolve "$t" > /dev/null 2>&1 && \
    echo "PASS $(basename $t)" || echo "FAIL $(basename $t)"
done

```

All previously-passing tutorials *must* still pass.

8 Adding a new thermodynamic model

Same factory pattern, with `ActivityModel` / `EquationOfState` / `VaporPressureModel` / `HeatCapacityModel` / `Phase` as the base.

Example — adding UNIQUAC as an activity model.

1. `src/thermo/activityCoefficient/UNIQUAC.{H,cpp}` inheriting `ActivityModel`. Override `gamma(x, T), readFromDict(...), type()`.
2. In `src/thermo/activityCoefficient/ActivityModel.cpp`:

```
#include "UNIQUAC.H"
//...
void ActivityModel::registerBuiltin() {
    reg("ideal", []{ return std::make_unique<Ideal>(); });
    reg("NRTL", []{ return std::make_unique<NRTL>(); });
    reg("Wilson", []{ return std::make_unique<Wilson>(); });
    reg("UNIQUAC", []{ return std::make_unique<UNIQUAC>(); }); // new
}
```

3. Use it in a tutorial's `thermoPackage`:

```
activityModel { model UNIQUAC; pairs (... ); }
```


9 Property operations (choupoProps)

`choupoProps` runs *property operations* rather than a flowsheet. Each inherits `PropertyOperation` (same explicit factory as everywhere else; `registerBuiltins()` is called from `src/propertyOps/PropertyOperation.cpp`). An op reads a `state` (T , P , composition), a `properties` list and an output block, and writes a CSV the GUI auto-plots. Built-ins: `propertyPoint`, `propertyScan1D`, `propertyScan2D`, `propertyScanTernary`, `fitParameters`, `vleConsistency`, `estimateComponent`. Property keywords live in `PropertyEvaluator.cpp`: per-component `Psat_<c>`, `gamma_<c>`, `y_eq_<c>`, `Cp_liquid_<c>` (each guarded — e.g. `Psat` on a component with no vapour-pressure model throws a clear error instead of dereferencing null), and mixture scalars `Z`, `v_molar`, `H_real`, `S_real`, `transport`, etc. `PropertyScan1D` wraps each evaluation in `try/catch`, writing `nan` on failure so one undefined component never aborts the whole scan.

propertyScanTernary — ternary phase diagrams by reuse. Walks the composition simplex ($x_1 + x_2 + x_3 = 1$) at fixed (T, P) and, at every node, calls the SAME solver the `isothermalFlash` unit op calls — so the diagram cannot disagree with a single flash. Three modes:

- `bubbleT` — a boiling-temperature *surface*: per node `BubblePoint::compute`; needs only the three vapour pressures (ideal binaries are fine), no liquid-liquid data.
- `lle` (default) — a *solubility* map: the liquid-liquid flash (`PhaseSet::LL`) gives a clean binodal + tie-lines, no spurious subcooled vapour (the extraction/decanter view).
- `vlle` — the full vapour + two-liquid classification (`PhaseSet::VLLE`).

Each simplex node is independent, so the scan is embarrassingly parallel: the optional `shard { k; n; }` block computes only rows with $i \bmod n = k$ (round-robin, balanced). The GUI fans $N = \text{cores} - 2$ shards across N WASM workers and merges the partial CSVs (the union of shards reproduces the full grid exactly). No physics is reimplemented — the op is a triangular loop + a classifier + a CSV writer over `solveCore/BubblePoint`.

Adding a property op. Mirror the recipe of §6: implement `src/propertyOps/MyOp.{H,cpp}` inheriting `PropertyOperation` (override `type()`, `run(dict,thermo,verbosity)`, optionally `diagnostics()`), then one `reg("myOp", ...)` line in `PropertyOperation::registerBuiltins()`. Rebuild the native binary AND the WASM (`make wasm-gui`) or the browser will not see the new op.

10 Adding components, materials, reactions

These are *data-driven* — no C++ changes needed.

10.1 Components

Choupo's component data is organised in two tiers:

- `data/standards/components/<name>.dat` — the **curated central catalogue** shipped with the simulator (46 components at v0.0.40). Treat this directory as read-only from a case author's point of view: changes here are reviewed edits to the standard library.
- `<case>/constant/components/<name>.dat` — **per-case overlays**, where a user adds bespoke or sample-specific data for a single case (a particular powder's sorption isotherm, a new membrane solute, a laboratory measurement). At load time **Database** overlays the per-case file *field-by-field* on top of the standard catalogue entry, so the overlay can extend or override individual fields without having to repeat the whole component definition.

The file format is identical in both tiers:

```
component <name>;
mw      78.11;           // g/mol
Tc      562.05;          // K
Pc      48.95;           // bar
omega   0.2103;
Tb      353.24;          // K
Hvap_Tb 30720.0;         // J/mol at Tb
Vliq    89.4e-6;         // m^3/mol (Wilson)

vaporPressure
{
    model    Antoine;
    A        4.01814;
    B        1203.835;
    C        -53.226;
}

idealGasHeatCapacity
{
    model    polynomial;
    coeffs   ( -31.368  0.4746  -3.107e-4  8.524e-8 );
    Tmin     200.0;
    Tmax     1500.0;
}

liquidHeatCapacity
{
    model    polynomial;
    coeffs   ( 162.94  -0.3493  9.51e-4  0.0 );
    Tmin     278.0;
    Tmax     500.0;
}

gibbsFormation
{
    dHf_298   82930.0;           // J/mol (NIST ideal-gas value)
    s_298     269.20;           // J/(mol*K) (third-law absolute)
    phase     gas;              // phase in which dHf_298 / s_298 are tabulated
}
```

Source the values from DIPPR / Reid-Prausnitz-Poling and *cite the source in a top-of-file comment* so they stay auditable.

The phase field (mandatory in `gibbsFormation`). The reference state for formation enthalpy is documented in the Theory Guide (chapter on the elements datum). The `phase` keyword tells the solver in *which phase* `dHf_298` is tabulated, so `Component::h_formation(T, targetPhase)` can build the right integration path through `Hvap` / `Hfus` jumps and the matching c_p leg. Three values are allowed:

phase	required data	typical species
gas	<code>idealGasHeatCapacity</code> , plus <code>HvapTb+Tc+Tb</code> if the liquid leg of any stream is needed	<i>the convention — all 45 standard volatiles + radicals; follows NIST / JANAF</i>
liquid	<code>liquidHeatCapacity</code> ; <code>HvapTb+Tc+Tb</code> if any stream goes through the gas leg	reserved for compounds whose Hf is published in the condensed datum (rare)
solid	<code>liquidHeatCapacity</code> (used as the dissolved-solute approximation $c_p^{\text{solid}} \approx c_p^{\text{liq}}$)	crystalline non-volatiles whose Hf cannot be referenced to gas because the compound never vaporises (sucrose)

The default is `gas` for backward compatibility with pre-v0.0.40 component files, but **new components should**

always declare it explicitly so the datum is unambiguous at the source of truth. A **solid** or **liquid** declaration without the matching c_p model throws a clear error from `Component::h_formation` the first time a stream containing the component is evaluated.

Add the new component name to the case's `thermoPackage.components (...)` list (or to a per-case `constant/components/<name>.dat` overlay if the data is sample-specific) and the new entry is picked up automatically — no recompile needed.

10.2 Materials

`data/materials/<name>.dat`:

```
material <name>;
rho      7900.0;      // kg/m^3
F_M      1.0;         // Guthrie material factor
sigma_y   138.0e6;    // Pa
maxT     700.0;       // K
maxP     50.0;        // bar
```

`MaterialRegistry::loadFrom()` slurps the directory on simulator start. No registration step needed.

10.3 Utilities

`data/standards/utilities/<name>.dat` catalogues a plant utility (steam header, cooling-water loop, hot-oil loop, refrigeration brine). The same registry pattern as materials and membranes: the `UtilityCatalogue::loadFrom(dataRoot)` call in each main slurps the directory at startup; the stream parser then resolves `utility <name>;` keys against it.

```
utility      steamLP;
tier         heating;           // heating | cooling
mechanism    condensation;      // condensation | sensible | evaporation

components   ( water );
state        saturatedVapour;
P            2.5 bar;
T_in         400.3 K;
T_out        400.3 K;           // condensate same T as steam

dutyPerKg    2186000.0;         // J/kg delivered (Hvap_water at T_sat)
cost         12.0;              // EUR/GJ delivered
costYear     2022;
description   "Low-pressure saturated steam header";
```

To add a new utility, drop a new `.dat` into the directory — no C++ change. When the utility loop carries a non-water fluid (oils, salts, brines), the `components` list points at a corresponding entry under `data/standards/components/`; the catalogue ships `dowthermA`, `hitecSalt` and `propyleneGlycol30` pseudo-components for that purpose. See the user guide for the case-author syntax `utility <name>;` and the `utilities` report that aggregates kg/h, MW, and EUR/h per category.

10.4 Reactions

In a case's `constant/reactions`:

```
reactions
{
    ethanol_to_water_test
    {
        stoichiometry
        {
            ethanol    -1.0;
            water       +1.0;
        }
        kinetics
        {
            type        powerLaw;
            A           1.0e6;      // s^-1
            E           80000.0;    // J/mol
        }
    }
}
```

```
        orders      { ethanol 1.0; }  
    }  
}
```

Referenced from a CSTR / PFR by name (`reaction ethanol_to_water_test;`).

11 Adding an outer driver / post-processor

11.1 Outer driver

src/outerDriver/MyDriver.{H,cpp} inherits OuterDriver. Override:

```
class MyDriver : public OuterDriver {
public:
    void readFromDict(const DictPtr&) override;
    int run() override;           // calls simulator_(flowsheetDict_) N times
};
```

The simulator functor and dict templates are injected by main.cpp:

```
auto driver = OuterDriver::New(outerDict);
driver->setSimulator      (simulate);           // SimulationResult(DictPtr)
driver->setFlowsheetDict  (flowsheetDict);      // mutable flowsheet template
driver->setThermoDict     (thermoDict);         // mutable thermoPackage
driver->run();
```

The simulator closure captures the thermo dict by `shared_ptr`, so any in-place `setScalarAtPath()` the driver makes on `thermoDict_` is picked up on the next `simulator_()` call.

Inside `run()`, you typically:

1. Mutate `flowsheetDict_>setScalarAtPath(...)` to change a sweep variable, OR `thermoDict_>setScalarAtPath(...)` to change a model parameter (e.g. NRTL `a_ij`).
2. Call `simulator_(flowsheetDict_)` → get a `SimulationResult`.
3. Extract KPIs / stream fields, accumulate residuals / responses.
4. Decide next step (LM / Nelder-Mead / GA / ...).
5. Eventually report and return 0 (converged) or 1 (failed).

Two concrete examples to read first:

- `src/outerDriver/SweepDriver.{H,cpp}` (~ 150 LOC) — 1-D sensitivity sweep.
- `src/outerDriver/FitBinaryPair.{H,cpp}` (~ 300 LOC) — Marquardt LM with FD Jacobian, mutates both flowsheet (composition) and thermo (pair params).

Register your driver in `OuterDriver::registerBuiltins()`.

11.2 Post-processor

src/postProcessing/MyPass.{H,cpp} inherits PostProcessor. Single method:

```
int run(SimulationResult&) override;
```

Append to `result.sizings` / `result.costs`, write CSVs, print to stdout. Register in `PostProcessor::registerBuiltins()`.

`PostProcessor::buildChain(postDict)` walks `postDict` top-level keys and instantiates one pass per recognised key. Add yours to the chain-builder in `PostProcessor.cpp`.

12 Adding a sizing model

`EquipmentSize` is a sub-factory used by `SizingPass`. Add `src/postProcessing/sizing/MyEquip.{H,cpp}` inheriting `EquipmentSize`:

```
class MyEquip : public EquipmentSize {
public:
    EquipmentSizing size(const std::string& unitName,
                        const SimulationResult& result,
                        const Material& material,
                        const DictPtr& designRules) override;
};
```

Use `result.streams.at(...)` and `result.kpis.at(unitName).at(...)` to get the process data, plus `designRules` for the design specs (pressure design, F_M , joint efficiency, ...). Fill `EquipmentSizing`:

```
EquipmentSizing d;
d.unitName      = unitName;
d.equipmentType = "myEquip";
d.material      = material.name;
d.values["V_R"] = ...;          // canonical size
d.values["t_wall"] = ...;       // m
d.values["weight"] = ...;       // kg
return d;
```

Register in `EquipmentSize::registerBuiltins()`.

The Guthrie costing model reads `d.equipmentType` to dispatch to the right correlation — so if your new equipment needs new cost coefficients, edit `src/postProcessing/costing/Guthrie.cpp` accordingly.

13 Coding conventions

13.1 Style

- C++ 17. `std::variant`, `std::optional`, `std::unique_ptr`, `std::filesystem` — yes. No raw `new` / `delete`. No `goto`.
- **Brace style:** Allman / open-brace on a new line for function definitions; Egyptian (`if (x) {`) for control flow.
- **Indent:** 4 spaces. No tabs anywhere.
- **Naming:** `snake_case` for locals, `camelCase` for methods, `PascalCase` for types, `UPPER_SNAKE` only for macros (we use exactly one: include guards via `#pragma once`).
- Compile cleanly with `-Wall -Wextra -Wpedantic` and fix anything that fires.

13.2 Includes

- Headers compile in isolation — every `.H` includes what it needs.
- Forward-declare aggressively in headers; pull full definitions only in `.cpp`.
- The build does not use precompiled headers.

13.3 Errors

- User-facing: throw `std::runtime_error` with a message a student can understand. Don't `assert` on user-data invariants.
- Programmer-facing (impossible state): `assert(...)` is fine.
- Never silently swallow errors. If a sub-step fails, surface it.

13.4 Comments

- Default to writing *no* comments. Identifiers should be obvious.
- Equations and references *deserve* a comment. Cite the source (Reid-Prausnitz-Poling page, DIPPR ID, Henley-Seader chapter).
- Top-of-file banner is preserved on every source — do not delete.

13.5 Header banner

Every `.H` and `.cpp` starts with:

```
/*-----*\
| Choupo                                     |
|                                           |
| Open-source chemical process simulator   |
|                                           |
| Copyright (C) 2026 Vítor Geraldés        |
| SPDX-License-Identifier: GPL-3.0-or-later |
|-----*/
Class
    Choupo::<ClassName>

Description
    <one-paragraph description>
/*-----*/
```

Don't omit this on new files — it doubles as licence notice.

14 Regression testing

Regression is run with `bin/runTests`. Run it before every change set; all previously-passing tutorials must still pass.

```
bin/runTests          # every tutorial (4 binaries + buildCode cases)
bin/runTests <case>   # one case
bin/runTests --record <case> # refresh that case's golden-master 'expected'
```

`runTests` does two checks, both stronger than a plain exit-code loop:

1. **NaN / inf guard** — **every case**. The structured result block is scanned for NaN or inf values; a hit FAILS the case. This is the check a bare `binary > /dev/null && PASS` loop *cannot* do: a solver can return NaN with exit code 0, and the old loop reported it green. That exact trap once hid a broken `membrane01` (NaN KPIs, exit 0) after a pressure-unit refactor.
2. **Reference KPIs** — **cases with an expected file**. Each line pins a KPI or stream value to its golden-master value with a relative tolerance; any deviation FAILS, printing `got X expected Y`.

Key idea. The `expected` file lives in the case directory. Format (`#` comments allowed):

# kind	name	key	expected	reitol
kpi	flash01	V_over_F	0.30398357314	1e-6
stream	vapor	F	0.00844398814	1e-6

`kind` is `kpi` (inside the named unit's KPIs) or `stream` (inside the named stream). Pin the numbers worth guarding; `-record` writes them from the current run once you trust it. A case with no `expected` still gets the NaN guard.

At, four canonical cases ship an `expected` (`flash01`, `cstr01`, `column01`, `membrane01`); the rest run under the smoke + NaN guard. Add an `expected` whenever a case's numbers are load-bearing.

15 Debugging

15.1 Debug build

```
make MODE=debug all
gdb build/linux64GccDebug/choupoSolve
(gdb) r tutorials/flash01_benzene_toluene
```

The debug build leaves a separate symlink `choupoSolve.debug` so the release binary stays usable in parallel.

15.2 Verbosity

`controlDict { verbosity 4; }` enables debug-level prints — every Newton iteration, every K -value evaluation, every Wegstein step. For LL flashes that misconverge, watch for stagnation: $g(x)$ should shrink monotonically near the root.

15.3 Sanitizers

Edit `make/compiler.mk` for the debug branch:

```
ifeq ($(MODE),debug)
    CXXFLAGS += -fsanitize=address,undefined -fno-omit-frame-pointer
    LDFLAGS  += -fsanitize=address,undefined
endif
```

Then rebuild. Useful when a unit-op produces NaN out of nowhere or when adding a new factory and a dispatch crashes.

15.4 The most common failure: NaN propagation

Symptoms: `verbosity 3` shows clean inputs, then $V/F = \text{nan}$ mid-flash. Almost always a $\log P_i^{\text{sat}}$ with $P_i \leq 0$ (Antoine outside its valid range) or an empty composition ($F = 0$). Add asserts where you suspect it.

16 Architecture deep-dive — the three layers

What you'll learn. The contract between the outer driver, the simulator core, and the post-processor. Read this twice before adding cross-layer logic.

```
+-----+
| Layer 1: OuterDriver (src/outerDriver/) |
| Sensitivity sweep, optimisation, parameter estimation, MC. |
| Owns: simulator_func, flowsheetDict_ (mutable). |
| Mutates flowsheetDict_ between calls via setScalarAtPath(). |
+-----+
| calls |
| v |
+-----+
| Layer 2: Simulator core |
| runSimulation(flowsheetDict,...) |
| -> ThermoPackage::readFromDict(thermoDict, db) |
| -> Flowsheet::solve(flowsheetDict, thermo, verbosity) |
| -> SimulationResult { streams, kpis, converged } |
| Pure function: same inputs -> same outputs. |
+-----+
| result |
| v |
+-----+
| Layer 3: PostProcessor (src/postProcessing/) |
| Sizing -> equipment dimensions (V_R, A, t_wall, weight,...) |
| Costing -> Guthrie/Turton with CEPCI, USD->EUR |
| (future) Reporting -> PDF / HTML / CSV bundles |
| Augments SimulationResult; never re-enters the simulator. |
+-----+
```

The contract:

- Layer 1 *never touches the simulator's internal state* — it only mutates the dict it owns and calls the functor.
- Layer 2 *is pure* — `runSimulation(d) == runSimulation(d)` for the same dict. This is what lets Layer 1 do finite-difference Jacobians and Levenberg-Marquardt cleanly.
- Layer 3 *never re-runs the simulator* — it only reads the result. If you need to re-run (e.g. for utility costing), do it via an outer driver.

Key idea. Rule of thumb: if a quantity can be computed from a converged stream table plus KPIs, it belongs in a post-processor. If it requires re-solving the flowsheet, it belongs in an outer driver.

17 Roadmap

Ordered by impact for the MCFT course and Vítor's research:

Target	Idea	Difficulty
(shipped)	Levenberg-Marquardt parameter-estimation driver (<code>fitBinaryPair</code>)	—
	Nelder-Mead optimisation driver (minimise cost / TAC)	easy
	LL flash via 2nd-order Michelsen TPD with multiple restarts	hard
	Solids: inert phase + filter / cyclone / dryer unit-ops	medium
	Crystalliser + PSD basics	hard
	NRTL distillation robustness (Naphtali-Sandholm)	medium-hard
v1.0	Equation-oriented sparse Newton for the full flowsheet	very hard
> v1	Membrane NF/RO unit op (Vítor's research bridge)	research-grade

Tech-debt items also welcome:

- Reference logs for every tutorial → in-tree regression diff.
- Reaction-kinetics library (Arrhenius, Langmuir-Hinshelwood, Michaelis-Menten) as a `Kinetics` base class.

18 Things to NOT do

What you'll learn. A short list, each item born from a real incident. Memorise it.

- Do **not** suggest a Python rewrite or wrapper. The course teaches numerical methods in C++; rewriting in Python would defeat the purpose.
- Do **not** add macro auto-registration. Explicit `registerBuiltin()` is non-negotiable (see §6).
- Do **not** add a heavy dependency — even a “small” one. Pedagogical visibility outweighs convenience.
- Do **not** split a binary by numerical strategy (**Foam*-style proliferation). Strategy is run-time. Splitting by *problem class* — the four binaries Solve / Batch / Ctrl / Props — is the deliberate exception, not a precedent for more.
- Do **not** bypass the dict format. Never use YAML, JSON, or TOML inside the case directory. The dict is the canonical wire format.
- Do **not** relitigate decisions in `CLAUDE.md` §10 without first reading what’s already been discussed and why.
- Do **not** merge code that breaks the regression, even if the failure is “obviously unrelated”.

19 Quick reference

Task	Command / file
Build release	<code>make all</code>
Build debug	<code>make MODE=debug all</code>
Clean	<code>make clean / make distclean</code>
Source env	<code>source etc/bashrc</code>
Run a case	<code>runCase tutorials/<name></code>
Run from inside case dir	<code>runCase</code>
List tutorials	<code>listCases</code>
List built-in unit-op types (in code)	<code>UnitOperation::availableTypes()</code>
Tweak a dict field programmatically	<code>dict->setScalarAtPath(path, v)</code>
Add a unit op	<code>§6</code>
Add a thermo model	<code>§7</code>
Add a component / material / reaction	<code>§8</code>
Add an outer driver / post-processor	<code>§9</code>
Add a sizing model	<code>§10</code>
Project conventions / AI rules	<code>CLAUDE.md</code>
User-facing how-to	<code>docs/userGuide.pdf</code>

Colophon. This manual was typeset in Latin Modern with the Choupo shared LaTeX preamble. Sources live in `docs/`; rebuild with `cd docs && make`.